SKILL-11

Command History and Pipeline Management

Objective

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.
To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

Program Description

This C program implements command history management and Unix pipeline execution. It stores commands in a fixed-size history buffer, retrieves and displays entries, validates the history, parses commands separated by '|', creates pipes and child processes, connects processes using dup2(), and executes commands using execvp().

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define HISTORY_SIZE 10
#define MAX_COMMANDS 10
#define COMMAND_SIZE 256

char history[HISTORY_SIZE][COMMAND_SIZE];
int history_count = 0;

void add_history(char *command)
{
    if (strlen(command) == 0) return;
    if (history_count < HISTORY_SIZE)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 1; i < HISTORY_SIZE; i++)
            strcpy(history[i - 1], history[i]);
        strcpy(history[HISTORY_SIZE - 1], command);
    }
}

void display_history()
{
    if (history_count == 0)
    {
        printf("History is empty.\n");
        return;
    }
    printf("\n=== Command History ===\n");
    for (int i = 0; i < history_count; i++)
        printf("%d  %s\n", i + 1, history[i]);
}

void retrieve_history(int index)
{
    if (index < 1 || index > history_count)
    {
        printf("Invalid history entry.\n");
        return;
    }
    printf("History Entry %d: %s\n", index, history[index - 1]);
}

int validate_history()
{
    if (history_count < 0 || history_count > HISTORY_SIZE) return 0;
    for (int i = 0; i < history_count; i++)
        if (strlen(history[i]) == 0) return 0;
```

```
    return 1;
}

int parse_pipeline(char *input, char *commands[])
{
    int count = 0;
    char *token = strtok(input, "|");
    while (token != NULL && count < MAX_COMMANDS)
    {
        while (*token == ' ') token++;
        if (strlen(token) == 0) return -1;
        commands[count++] = token;
        token = strtok(NULL, "|");
    }
    if (token != NULL) return -1;
    return count;
}

void execute_pipeline(char *commands[], int count)
{
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return;
        }
    }

    for (int i = 0; i < count; i++)
    {
        pids[i] = fork();
        if (pids[i] == -1)
        {
            perror("fork");
            return;
        }

        if (pids[i] == 0)
        {
            if (i > 0) dup2(pipes[i - 1][0], STDIN_FILENO);
            if (i < count - 1) dup2(pipes[i][1], STDOUT_FILENO);

            for (int j = 0; j < count - 1; j++)
            {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            char *args[COMMAND_SIZE];
            int argc = 0;
            char *token = strtok(commands[i], " \t");

            while (token != NULL)
            {
                args[argc++] = token;
                token = strtok(NULL, " \t");
            }

            args[argc] = NULL;
            if (argc == 0) exit(1);

            execvp(args[0], args);
            perror("exec");
            exit(1);
        }
    }

    for (int i = 0; i < count - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    for (int i = 0; i < count; i++)
        waitpid(pids[i], NULL, 0);
}

void pipeline_test(char *input)
{
    char copy[COMMAND_SIZE];
    char *commands[MAX_COMMANDS];

    strcpy(copy, input);
    int count = parse_pipeline(copy, commands);

    if (count <= 0)
    {
```

```
        printf("Invalid pipeline layout.\n");
        return;
    }

    printf("\n=== Pipeline Structure ===\n");
    for (int i = 0; i < count; i++)
        printf("Process %d: %s\n", i + 1, commands[i]);

    printf("Pipeline commands: %d\n", count);
    printf("Execution order: ");

    for (int i = 0; i < count; i++)
    {
        printf("%d", i + 1);
        if (i < count - 1) printf(" -> ");
    }

    printf("\n");
    printf("Pipeline layout validated successfully.\n");
    printf("Executing pipeline...\n");
    execute_pipeline(commands, count);
    printf("Pipeline execution completed.\n");
}

int main()
{
    char input[COMMAND_SIZE];

    printf("=== Skill-8: Command History and Pipeline Management ===\n");

    while (1)
    {
        printf("\nshell> ");

        if (fgets(input, sizeof(input), stdin) == NULL) break;
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        if (strcmp(input, "history") == 0)
        {
            display_history();
            continue;
        }

        if (strncmp(input, "retrieve ", 9) == 0)
        {
            retrieve_history(atoi(input + 9));
            continue;
        }

        if (strchr(input, '|') != NULL)
        {
            add_history(input);
            pipeline_test(input);
            continue;
        }

        add_history(input);
        printf("Command stored in history.\n");

        if (validate_history())
            printf("History consistency: Valid\n");
        else
            printf("History consistency: Invalid\n");
    }

    return 0;
}
```

Compilation and Execution

```
gcc skill-11.c -o skill-11
./skill-11
```

Tested Input

```
shell> pwd
shell> ls
shell> whoami
shell> history
shell> retrieve 2
shell> ls | wc -l
shell> exit
```

Observed Output

```
Command stored in history.
History consistency: Valid

=== Command History ===
1  pwd
2  ls
3  whoami
```

Result

The program demonstrates command storage, history capacity handling, retrieval, display, consistency validation, pipeline parsing, process creation, inter-process communication through pipes, execution ordering, and command execution.

Terminal Evidence

```
dmvs@DMVS:/mnt/d/OS/2520030106_Skill$ cd skill11
dmvs@DMVS:/mnt/d/OS/2520030106_Skill/skill11$ gcc skill-11.c -o skill-11
dmvs@DMVS:/mnt/d/OS/2520030106_Skill/skill11$ ./skill-11
=== Skill-11: Command History and Pipeline Management ===

shell> pwd
Command stored in history.
History consistency: Valid

shell> ls
Command stored in history.
History consistency: Valid

shell> whoami
Command stored in history.
History consistency: Valid
```